

IEC 61131-3 & ISPSOft — A Primer for ESP-Flow

Audience: the ESP-Flow development team. **Purpose:** explain, from zero, the concepts our data model and compiler must implement — scan cycle, data types, variables, POUs, tasks, and the five languages — and show how Delta's ISPSOft presents each one in its UI. **How to read it:** sections 1–3 are the foundation and are worth reading in order. Section 4 covers the languages. Section 5 is ISPSOft-specific. Section 6 maps every concept onto our own codebase.

1. What a PLC actually is

1.1 The scan cycle

A PLC is a computer that runs one program, forever, in a fixed repeating loop called the **scan cycle**:

```
+-----+
| 1. READ INPUTS    sample every physical |
|                   input pin into memory |
| 2. EXECUTE LOGIC  run the user program   |
|                   against that memory    |
| 3. WRITE OUTPUTS  copy result memory out |
|                   to physical output pins|
| 4. HOUSEKEEPING   comms, diagnostics,    |
|                   watchdog                |
+-----+
      ^                               |
      +-----+
              repeat forever
```

If you have written an Arduino sketch, the shape is familiar — `loop()` runs forever. The difference is discipline: a PLC **never** reads a pin in the middle of the logic.

1.2 The process image — why step 1 and step 3 are separate

The memory copy of the inputs is the **input process image**; the copy of the outputs is the **output process image**. Logic reads and writes *only* the image, never the hardware.

This matters. Suppose input `X0` is a vibrating limit switch and your program tests it in three places. If each test read the pin directly, one scan could see `TRUE`, `FALSE`, `TRUE` and produce logically impossible behaviour. With a process image, `X0` is sampled once and holds one value for the entire scan. The program becomes deterministic and reasoning about it is sound.

Consequence for us: **the generated C++ must call `digitalRead()` only in phase 1 and `digitalWrite()` only in phase 3**. Everything in between touches RAM.

1.3 Scan time

One full trip round the loop is the **scan time** — typically 1–20 ms. Two rules follow:

- An input pulse shorter than one scan can be missed entirely (hence edge-detection blocks and hardware interrupts).
- An output responds no faster than one scan after its input changes.

Since we transpile to native C++ on an ESP32 at 240 MHz our scans will be fast, but the *guarantee* users expect is consistency, not speed.

2. What IEC 61131-3 is

IEC 61131 is the international standard for programmable controllers, published in parts:

Part	Subject
61131-1	General information, definitions
61131-2	Equipment requirements and tests (the hardware)
61131-3	Programming languages ← this is our standard
61131-5	Communications
61131-9	Single-drop digital interface (IO-Link)

Editions of Part 3 that matter:

- **2nd edition (2003)** — the version most products, including much of ISPSOFT, actually implement.
- **3rd edition (2013)** — adds object-oriented extensions (classes, methods, interfaces, inheritance), extra data types, and **deprecates IL** (Instruction List).

Part 3 has two halves, and this split is exactly how our codebase should be organised:

1. **Common elements** — the type system, variables, POU's, tasks, configuration. Language-independent. *This is the part everyone skips and then regrets skipping.*
2. **Programming languages** — LD, FBD, ST, IL, SFC. Five syntaxes over one shared model.

PLCopen is the vendor-neutral trade organisation around the standard. Its TC6 working group publishes the **PLCopen XML** interchange format — the file format we chose for import/export — plus the widely used Motion Control function block library.

3. Common elements — the vocabulary

3.1 Data types

Every value in an IEC program has a declared type. The **elementary types**:

Category	Types	Notes
Boolean	BOOL	1 bit, TRUE / FALSE
Signed integer	SINT INT DINT LINT	8 / 16 / 32 / 64 bit
Unsigned integer	USINT UINT UDINT ULINT	8 / 16 / 32 / 64 bit
Bit strings	BYTE WORD DWORD LWORD	8 / 16 / 32 / 64 bit — for bitwise work, not arithmetic
Real	REAL LREAL	32 / 64 bit float
Duration	TIME	literal T#5s, T#1h30m
Date & time	DATE, TIME_OF_DAY (TOD), DATE_AND_TIME (DT)	
Text	STRING WSTRING	single-byte / wide characters

The 3rd edition adds CHAR, WCHAR, LTIME and friends.

INT and WORD are both 16 bits, and the distinction is deliberate: INT is a number you do arithmetic on, WORD is a bag of 16 bits you mask and shift. The standard will not silently convert between them.

Derived types are declared by the user in a TYPE . . . END_TYPE block:

```

TYPE
    (* enumeration *)
    MachineState : (Idle, Starting, Running, Stopping, Faulted);

    (* subrange - the compiler enforces the bounds *)
    Percent : INT (0..100);

    (* array *)
    RecipeTemps : ARRAY [1..10] OF REAL;

    (* structure *)
    Motor : STRUCT
        Running    : BOOL;
        Speed      : REAL;
        FaultCode  : INT;
    END_STRUCT;

    (* alias *)
    Temperature : REAL;
END_TYPE

```

There are also **generic types** (ANY, ANY_NUM, ANY_INT, ANY_BIT...) used to declare overloaded standard functions such as ADD, which accepts any numeric type.

3.2 Variables

This is the concept to internalise first. A variable is a named, typed piece of controller memory. You write MotorRunning, not M100.

Older PLC programming (and Delta's own DVP heritage) works with **device addresses**: X0 is the first input, M100 is auxiliary relay 100, D50 is data register 50. It works, but the program reads like assembly language and nothing tells you that M100 is a boolean while D50 is a 16-bit integer.

IEC 61131-3 puts a variable layer on top: you declare names with types, and you *optionally* say where a name lives in hardware.

Variables are declared in blocks, and the block determines the variable's role:

Block	Meaning
VAR ... END_VAR	Local to this POU. Private working memory.
VAR_INPUT	A parameter passed <i>into</i> a function or function block.

Block	Meaning
VAR_OUTPUT	A result passed <i>out</i> of a function block.
VAR_IN_OUT	Passed by reference — the POU may modify the caller's variable.
VAR_TEMP	Scratch memory, wiped at the start of every call.
VAR_GLOBAL	Declared once at configuration/resource level, visible project-wide.
VAR_EXTERNAL	"I intend to use a VAR_GLOBAL declared elsewhere." Makes the dependency explicit.
VAR_CONFIG	Assigns concrete addresses to partly-specified variables in program instances.

And qualifiers modify them:

Qualifier	Meaning
RETAIN	Value survives a power cycle (battery-backed / flash-stored memory).
NON_RETAIN	Explicitly cleared on restart.
CONSTANT	Read-only after initialisation.
AT %...	Located at a specific hardware address (see 3.3).

```

VAR
    StartButton    : BOOL;                (* plain local variable *)
    Setpoint       : REAL := 72.5;        (* with an initial value *)
    CycleCount     : DINT;
END_VAR

VAR RETAIN
    TotalParts     : DINT;                (* survives power loss *)
END_VAR

VAR CONSTANT
    MaxPressure    : REAL := 150.0;
END_VAR

```

Initial values matter. Every variable has a defined initial value — either the one you wrote, or the type's default (0, FALSE, T#0s, empty string). There is no uninitialised memory in a conforming IEC program.

3.3 Located variables and direct addressing

Some variables *must* correspond to real hardware. IEC gives them a **direct address**:

```

% I X 0 . 0
| | | +----- position (hierarchical, dot-separated)
| | +----- size prefix
| +----- location prefix
+----- introduces a direct address

```

Location prefix

Meaning

I	Input
Q	Output (o was avoided because it looks like zero)
M	Internal memory

Size prefix

Meaning

(none)	single bit
X	single bit (explicit)
B	byte, 8 bit
W	word, 16 bit
D	double word, 32 bit
L	long word, 64 bit

Examples: %IX0.0 (input bit 0 of word 0), %QX2.7, %MW10 (internal 16-bit word 10), %ID4.

You bind a variable to an address with AT:

```
VAR_GLOBAL
    EmergencyStop AT %IX0.0 : BOOL;
    ConveyorMotor AT %QX0.1 : BOOL;
    OvenTemp      AT %IW4   : INT;
END_VAR
```

Now the program says IF EmergencyStop THEN, and only this one declaration knows it is physical input bit 0.0.

Rewire the panel, change one line.

A variable with AT is **located**; one without is **unlocated** (the runtime places it wherever). AT %I* declares a **partly located** variable — "this is an input, the address is assigned later at configuration time".

3.4 POU — Program Organization Unit

A **POU** is the unit of code: the standard's equivalent of a function, class, or module. Every POU has two parts:

```
+-----+
| Declaration part      | <- the VAR blocks
+-----+
| Body                  | <- written in LD, FBD, ST, IL or SFC
+-----+
```

There are exactly three kinds, and the difference between them is **memory**:

FUNCTION — no memory

Same inputs always produce the same output. Nothing persists between calls. Returns a single value.

```

FUNCTION CelsiusToF : REAL
VAR_INPUT
    Celsius : REAL;
END_VAR
    CelsiusToF := Celsius * 1.8 + 32.0;
END_FUNCTION

```

Standard functions include ADD, SUB, MUL, DIV, MOD, ABS, SQRT, SIN, AND, OR, XOR, NOT, SHL, SHR, SEL, MAX, MIN, LIMIT, MUX, the comparison operators, and type conversions such as INT_TO_REAL.

FUNCTION_BLOCK — has memory

A function block keeps **instance state** between calls. This is the crucial one, because timers, counters, edge detectors and PID loops all need to remember something from the previous scan.

A function block is a *type*. To use it you declare an **instance** — exactly like declaring an object of a class:

```

FUNCTION_BLOCK MotorStarter
VAR_INPUT
    Start : BOOL;
    Stop  : BOOL;
END_VAR
VAR_OUTPUT
    Running : BOOL;
END_VAR
VAR
    Latched : BOOL;      (* private state, persists across scans *)
END_VAR
    Latched := (Latched OR Start) AND NOT Stop;
    Running := Latched;
END_FUNCTION_BLOCK

```

Used as:

```

VAR
    Pump1 : MotorStarter;  (* instance 1 - its own Latched  *)
    Pump2 : MotorStarter;  (* instance 2 - a separate Latched *)
END_VAR

Pump1(Start := Btn1, Stop := EStop);
Pump2(Start := Btn2, Stop := EStop);
IF Pump1.Running THEN ...  (* outputs read via dotted access *)

```

Two instances, two independent memories. **Every function block instance is itself a variable and must appear in the variable table** — this is why our editor cannot treat a timer as just another symbol dropped on a rung.

PROGRAM — the top level

A `PROGRAM` is like a function block, but it is the outermost unit: it may access global and located variables directly, and it is the only POU kind that can be **assigned to a task**.

Calling rules

Caller	May call
FUNCTION	functions only
FUNCTION_BLOCK	functions and function blocks
PROGRAM	functions and function blocks

Recursion is forbidden. A PLC program must have a computable worst-case execution time.

3.5 The standard function block library

Every conforming system provides these. Our compiler and C++ runtime must implement them:

Block	Purpose
SR / RS	Set-dominant / reset-dominant bistable (latch)
R_TRIG / F_TRIG	Rising / falling edge detection
TON	On-delay timer — <code>Q</code> goes TRUE after <code>IN</code> has been TRUE for <code>PT</code>
TOF	Off-delay timer — <code>Q</code> stays TRUE for <code>PT</code> after <code>IN</code> goes FALSE
TP	Pulse timer — fixed-width pulse on a rising edge
CTU / CTD / CTUD	Count up / down / up-down

A `TON` in Structured Text:

```
VAR
    DelayTimer : TON;
END_VAR

DelayTimer(IN := StartSignal, PT := T#5s);
Motor      := DelayTimer.Q;      (* TRUE 5 seconds after StartSignal *)
Elapsed    := DelayTimer.ET;     (* elapsed time, readable any scan *)
```

3.6 Configuration, Resource and Task — how the program gets scheduled

This layer answers "*when* does my code run?", and it is the one most often ignored by hobby tools.


```

CONFIGURATION    the whole control system (one PLC or a distributed set)
+-- RESOURCE    one processing unit / CPU
    +-- TASK      a scheduling rule
    |    +-- PROGRAM instance bound to that task
    +-- VAR_GLOBAL    variables shared across this resource

```

A task has:

- **INTERVAL** — run cyclically, e.g. every 10 ms. This is the ordinary scan.
- **SINGLE** — run once when a boolean event becomes TRUE (an interrupt).
- **PRIORITY** — which task wins when two are ready. 0 is highest.

```

CONFIGURATION PlantCell
RESOURCE CPU_1 ON ESP32
    TASK FastScan (INTERVAL := T#10ms, PRIORITY := 1);
    TASK SlowScan (INTERVAL := T#500ms, PRIORITY := 3);

    PROGRAM Safety WITH FastScan : SafetyLogic;
    PROGRAM Reports WITH SlowScan : ReportingLogic;
END_RESOURCE
END_CONFIGURATION

```

Two programs, two different rates, defined priorities. On an ESP32 this maps naturally onto FreeRTOS tasks or hardware timers — which is why we need this model *before* writing the code generator, not after.

3.7 Scope

- **Global** — declared in `VAR_GLOBAL` at configuration or resource level; visible to any POU that declares `VAR_EXTERNAL`.
- **Local** — declared inside a POU; invisible outside it.
- **Instance** — the memory belonging to one function block instance, reached with dotted access (`Pump1.Running`).

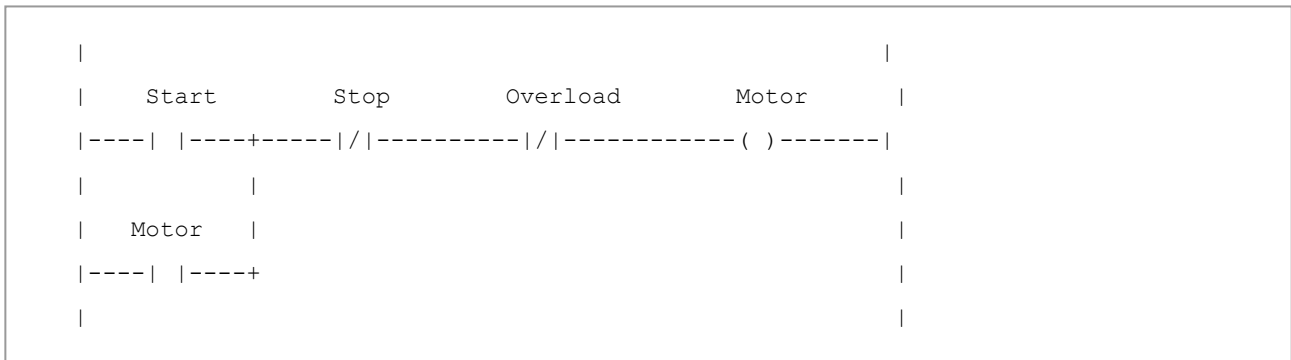
Local wins over global when names collide, and the standard requires that shadowing be at least diagnosable.

4. The five languages

All five compile to the same underlying semantics. They differ in how the logic is *written*.

4.1 LD — Ladder Diagram (graphical) — our first priority

LD draws logic as an electrical relay circuit between two vertical **power rails**. Power flows from the left rail, through contacts, into coils on the right.



That is the classic **seal-in circuit**: pressing Start energises Motor; the Motor contact in the parallel branch keeps it energised after Start is released; Stop or Overload breaks the path.

Elements:

Symbol	Name	Passes power when
--\ / --	Normally open contact (NO)	its variable is TRUE
--\ /\ --	Normally closed contact (NC)	its variable is FALSE
--\ P \ --	Positive transition contact	its variable changed FALSE→TRUE this scan
--\ N \ --	Negative transition contact	its variable changed TRUE→FALSE this scan
-- () --	Coil	assigns power-flow state to its variable
-- (/) --	Negated coil	assigns the inverse
-- (S) --	Set coil (latch)	sets TRUE, stays TRUE until reset
-- (R) --	Reset coil (unlatch)	sets FALSE

The two structural rules that drive our compiler:

- **Series = AND**. Contacts in a row must all conduct or no power reaches the coil.
- **Parallel = OR**. A branch offers an alternative path.

A horizontal block of logic ending in one or more outputs is a **rung**; IEC calls it a **network**. Networks execute strictly **top to bottom**, and within a network, **left to right**. That ordering is semantic, not cosmetic: if network 1 writes M0 and network 5 reads it, network 5 sees this scan's value; the reverse ordering would see last scan's.

Function blocks appear in rungs as boxes wired into the power flow, usually with **EN/ENO**: EN (enable in) gates whether the block executes at all, ENO (enable out) reports success and lets you chain blocks.

Contacts *read* variables; coils *write* them. Two coils writing the same variable in different networks is the classic **double coil** error — the second silently wins. Every serious IDE flags it, and ours must too.

4.2 FBD — Function Block Diagram (graphical)

Boxes with input pins on the left and output pins on the right, joined by wires. Reads like a signal-flow or circuit diagram, and suits analogue processing, maths chains and PID far better than ladder does.

```

Start ---+
      |   +-----+           +-----+
      +---+ AND +-----+ IN   Q  +---- Motor
NotStop---+ +-----+           |   TON   |
              T#5s ---+ PT   ET +---- Elapsed
                      +-----+

```

Execution order follows data dependency; where that is ambiguous the tool assigns and displays an explicit order number.

4.3 ST — Structured Text (textual)

A Pascal-like high-level language, and the most expressive of the five. Anything involving loops, string handling or non-trivial arithmetic belongs here.

```

IF Temperature > Setpoint + Hysteresis THEN
    Heater := FALSE;
ELSIF Temperature < Setpoint - Hysteresis THEN
    Heater := TRUE;
END_IF;

CASE Machine OF
    0:    Status := 1;
    1:    Status := 2;
    2, 3: Status := 3;
ELSE
    Status := 0;
END_CASE;

FOR i := 1 TO 10 DO
    Total := Total + Readings[i];
END_FOR;

WHILE NotAtTarget DO
    Position := Position + Step;
END_WHILE;

```

Assignment is :=; comparison is =; comments are (* ... *), with // line comments added in the 3rd edition. Control constructs: IF/ELSIF/ELSE, CASE, FOR, WHILE, REPEAT/UNTIL, EXIT, RETURN.

Unbounded WHILE loops are dangerous in a PLC — an accidental infinite loop trips the scan watchdog and faults the controller.

4.4 IL — Instruction List (textual, deprecated)

An accumulator-based, assembly-like language:

```
LD   Start
OR   Motor
ANDN Stop
ST   Motor
```

Deprecated in the 3rd edition. Worth supporting only for importing legacy code — not worth building an editor for.

4.5 SFC — Sequential Function Chart (graphical)

SFC is different in kind: it describes *sequence* rather than *logic*. It is the right tool for "do step 1, wait for a condition, then do step 2" — batch processes, machine start-up, state machines.

```
+=====+
|  Init   |          <- initial step (double border)
+-----+-----+
|         |
+-----+-----+ StartButton AND NOT Fault    <- transition
|         |
+-----+-----+
|  Fill   |---- N: FillValve                    <- step with an action
+-----+-----+
|         |
+-----+-----+ LevelHigh
|         |
+-----+-----+
|  Heat   |---- N: Heater
+-----+-----+
```

Made of:

- **Steps** — states. Exactly one initial step. A step is active or inactive; `Step.X` is its activity bit and `Step.T` its elapsed active time.
- **Transitions** — boolean conditions guarding the move from one step to the next.
- **Actions** — the work done, attached to steps with a **qualifier** that controls its timing:

Qualifier	Behaviour
N	Non-stored — active while the step is active
S	Set (stored) — stays active after the step deactivates
R	Reset — cancels a stored action
P	Pulse — executes once

Qualifier	Behaviour
L	Time limited
D	Time delayed
SD / DS / SL	Stored-and-delayed combinations

Branching comes in two flavours: **selective divergence** (choose one path — an OR) and **simultaneous divergence** (run several paths in parallel, drawn with a double line — an AND).

SFC is formally a *structuring element*: the actions inside its steps are written in LD, FBD, ST or IL.

5. How Delta ISPSoft presents all of this

ISPSoft is Delta's IEC-compliant IDE for the AH/AS series. (The older DVP series uses WPLSoft, which is device-address-only and not IEC-structured.) Understanding its UI is understanding our target.

5.1 The project tree

A typical ISPSoft project shows, roughly:

```
Project
+-- Device Comment      comments attached to raw device addresses
+-- HWCONFIG             CPU, power supply, I/O module rack layout
+-- NWCONFIG             network / fieldbus topology
+-- Global Symbols      project-wide variables          <- VAR_GLOBAL
+-- Programs            PROGRAM POU's
|   +-- Prog0   (Ladder)
|   +-- Prog1   (ST)
+-- Function Blocks      FUNCTION_BLOCK POU's
+-- Tasks                cyclic and interrupt tasks
+-- Device Monitor Table live watch tables
```

Map that onto section 3 and every node has a standard concept behind it.

5.2 Symbols vs devices — the point of confusion worth resolving

Delta hardware addresses memory by **device**:

Device	Meaning	Type
X	Physical input relay	bit
Y	Physical output relay	bit
M	Auxiliary (internal) relay	bit
SM	Special auxiliary relay (system flags)	bit

Device	Meaning	Type
S	Step relay (used by SFC)	bit
T	Timer	bit + value
C	Counter	bit + value
HC	High-speed counter	32-bit
D	Data register	word
SR	Special data register	word
E	Index register	word

Exact ranges and bit-addressing format vary by series — AH/AS use an `X0.0 word.bit` form, older DVP uses octal `X0-X7, X10...` Always check the series programming manual before hard-coding ranges.

On top of these, ISPSoft has a **symbol table** — Global Symbols and per-POU Local Symbols — with columns for Identifier, Address, Data Type, Initial Value and Comment. That table is the IEC variable declaration:

ISPSoft column	IEC 61131-3 equivalent
Identifier	variable name
Address (optional)	AT <code>%IX0.0</code> — the located address
Data Type	BOOL, INT, REAL...
Initial Value	<code>:= value</code>
Comment	documentation

A symbol with an address is a located variable; a symbol without one is unlocated and the compiler allocates it. The correspondence:

Delta device IEC direct address

X	<code>%I</code>
Y	<code>%Q</code>
M, D	<code>%M</code>

5.3 The working cycle in ISPSoft

1. **Configure hardware** (HWCONFIG) — CPU and I/O modules, which fixes the available `x/y` addresses.
2. **Declare symbols** — global first, then local per POU.
3. **Write POU**s — ladder editor with networks, an ST editor, function blocks.
4. **Assign to tasks** — cyclic task 0 by default; interrupt tasks for fast events.
5. **Compile** — syntax and semantic checks; produces downloadable code.
6. **Download** to the PLC over USB/Ethernet.
7. **Monitor online** — the ladder is redrawn live, conducting contacts highlighted, current values shown next to every symbol.
8. **Force / set values** — override an input or output for commissioning.
9. **Online edit** — modify the running program without stopping the machine.

Steps 7–9 are where a PLC IDE earns its keep, and they are the hardest part of our project. Note that our transpile-and-rebuild decision makes online edit (step 9) effectively impossible and step 6 a 20–60 second operation; live monitoring (7) and forcing (8) remain fully achievable over our WebSocket link.

6. How each concept lands in ESP-Flow

IEC concept	In our app	Lives in
Process image	<code>bool X[], Y[]; int16_t D[]</code> arrays in the runtime	firmware/ C++ runtime
Scan cycle	<code>readInputs(); runLogic(); writeOutputs();</code> in <code>loop()</code>	generated <code>main.cpp</code>
Data type	<code>DataType</code> enum + type checker	frontend/src/core/, Django compiler
Variable	Symbol-table UI + <code>Variable</code> model (name, type, scope, retain, optional address)	frontend <code>core/</code> , Django models
Located variable AT <code>%IX0.0</code> RETAIN	The GPIO ↔ address mapping table	ui/features/hardware-config
POU	ESP32 NVS / Preferences storage Project-tree nodes: Programs, Function Blocks, Functions	C++ runtime Django models + frontend tree
Function block instance	An entry in the variable table whose type is an FB; generates a C++ struct instance	compiler + runtime
Network / rung	Our existing <code>Network</code> model (grid + coils)	<code>core/models/network.ts</code>
Contact / coil	<code>ContactInstruction</code> / <code>CoilInstruction</code>	<code>core/models/instruction.ts</code>
Series / parallel	<code>verticalLink</code> resolution → AND/OR nodes in the AST	rung→AST compiler
Task	Task configuration → FreeRTOS task or timer interval	Django models + codegen
Standard FBs (TON, CTU...)	Hand-written C++ classes in our runtime library, instantiated by codegen	firmware/lib/
Compile	Django: JSON → AST → validate → C++ → PlatformIO	backend/
Online monitoring	WebSocket streaming the process image; canvas highlights conducting paths	Channels + ui/features/monitoring
PLCopen XML	Import/export serializer	backend/

6.1 What the generated C++ will look like

```

// --- Process image -----
bool    X[32];        // inputs
bool    Y[32];        // outputs
bool    M[256];       // internal bits
int16_t D[512];       // data registers

// --- Located variable bindings (from hardware config) ---
const uint8_t PIN_X0 = 34;    // EmergencyStop  AT %IX0.0
const uint8_t PIN_Y0 = 26;    // ConveyorMotor  AT %QX0.0

// --- Function block instances (from the variable table) ---
TON DelayTimer;

void setup() {
    pinMode(PIN_X0, INPUT);
    pinMode(PIN_Y0, OUTPUT);
    restoreRetentiveMemory();
}

void loop() {
    uint32_t scanStart = millis();

    // 1. READ INPUTS
    X[0] = digitalRead(PIN_X0);

    // 2. EXECUTE LOGIC    (network 1: X0 AND NOT M0 -> Y0)
    Y[0] = X[0] && !M[0];

    //    (network 2: TON instance)
    DelayTimer.update(X[0], 5000);
    M[1] = DelayTimer.Q;

    // 3. WRITE OUTPUTS
    digitalWrite(PIN_Y0, Y[0]);

    // 4. HOUSEKEEPING
    serviceMonitoring();
    scanTime = millis() - scanStart;
}

```

Every line of that is a direct consequence of a concept above. The scan phases come from §1, the arrays from the process image, the pin constants from located variables, `TON` from the standard library, and `DelayTimer` from the variable table.

7. Glossary

Term	Definition
Scan cycle	The read-execute-write-housekeep loop a PLC repeats forever
Process image	The memory copy of inputs/outputs the program works against
Scan time	Duration of one scan cycle
POU	Program Organization Unit — a <code>FUNCTION</code> , <code>FUNCTION_BLOCK</code> or <code>PROGRAM</code>
Function	Stateless POU returning one value
Function block	POU with persistent instance memory; must be instantiated
Instance	One named copy of a function block, with its own memory
Program	Top-level POU; the only kind assignable to a task
Variable	A named, typed piece of memory
Located variable	A variable bound to a hardware address with <code>AT %...</code>
Direct address	<code>%IX0.0</code> style hardware reference
Symbol	Delta/ISPSoft's word for a variable
Device	Delta's raw address (<code>X0</code> , <code>M100</code> , <code>D50</code>)
Retentive	Memory that survives power loss (<code>RETAIN</code>)
Configuration	The whole control system in the IEC model
Resource	One CPU within a configuration
Task	A scheduling rule (interval or event) binding programs to execution
Network / rung	One horizontal block of ladder logic
Power flow	The boolean state travelling left-to-right through a rung
Contact	Reads a boolean variable, gates power flow
Coil	Writes power-flow state to a boolean variable
Double coil	Error: the same variable written by two coils
EN / ENO	Enable-in / enable-out pins gating and chaining block execution
Step / transition / action	The three SFC elements
Action qualifier	<code>N</code> , <code>S</code> , <code>R</code> , <code>P</code> , <code>L</code> , <code>D...</code> — controls when an SFC action runs
PLCopen	Vendor-neutral organisation; publishes the TC6 XML interchange format
AST	Abstract Syntax Tree — the compiler's structured representation of logic

8. Where to read more

- **IEC 61131-3:2013** — the standard itself, from the IEC webstore (paid).
- **PLCopen** — plcopen.org, for the TC6 XML specification and reference libraries.
- **Delta ISPSoft User Manual** and the **AH/AS Series Programming Manual** — Delta's own documentation of the device table, instruction set and symbol behaviour. The closest thing we have to a product spec.

- **"Programming Industrial Control Systems Using IEC 1131-3"** (Lewis) — the standard textbook treatment.
- **OpenPLC, Beremiz** — open-source IEC 61131-3 implementations worth reading for compiler structure and runtime design.

